1. Dependencia

Tu controlador utiliza el servicio para realizar operaciones.

Java

```java
public class EncuestaController {

    private EncuestaService service;

    public EncuestaController() {
        service = new EncuestaService();
    }

    public void registrarEncuesta() {
        service.registrarEncuesta();
    }
}
```

UML:

Plain Text

```
EncuestaController - - - - > EncuestaService
```

Porque EncuestaController necesita a EncuestaService.

2. Asociación

Una encuesta está relacionada con un cliente.

Java

```java
public class Encuesta {

    private Cliente cliente;
    private int calificacion;
    private String comentario;
}
```

UML:

Plain Text

```
Encuesta -------- Cliente
```

Porque cada encuesta pertenece a un cliente.

3. Agregación

Tu repositorio guarda muchas encuestas.

Java

```java
public class EncuestaRepository {

    private List<Encuesta> encuestas;
}
```

UML:

Plain Text

```
EncuestaRepository ◇——— Encuesta
```

El repositorio contiene encuestas.

Si el repositorio desaparece, las encuestas podrían seguir existiendo.

4. Composición

Ejemplo.

Java

```java
public class Encuesta {

    private Respuesta respuesta;
}
```

y la respuesta sólo existe dentro de la encuesta.

UML:

Plain Text

```
Encuesta ◆——— Respuesta
```

La respuesta no tiene sentido sin la encuesta.

5. Herencia

Sistema maneja roles.

Java

```java
public class Usuario {

    private String usuario;
    private String password;
}
```

Java

```java
public class Cliente extends Usuario {
}
```

Java

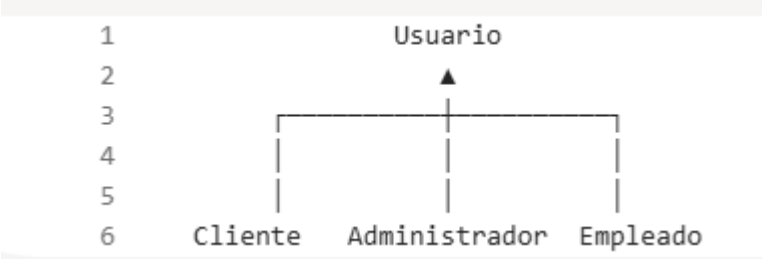
```java
public class Administrador extends Usuario {
}

```

Java

```java
public class Empleado extends Usuario {
}
```

UML:



Cliente, Administrador y Empleado heredan de Usuario.

El diagrama quedaría así:

```
                    Usuario
                       ▲
                       |
            ┌──────────┴──────────┐
            |                     |
       Cliente               Administrador


LoginView ---------> Usuario

MainMenu ---------> EncuestaController

EncuestaView ------> EncuestaController

EncuestaController ------> EncuestaService

EncuestaService ------> EncuestaRepository

EncuestaRepository ◇------ Encuesta

Encuesta -------- Cliente
```

Dibujo en Draw.io

```
EncuestaView
      |
      | dependencia
      ▼
EncuestaController
      |
      | dependencia
      ▼
EncuestaService
      |
      | asociación
      ▼
EncuestaRepository
      |
      | agregación
      ▼
Encuesta
      |
      | asociación
      ▼
Cliente
```

Regla rápida para identificar relaciones en el código

Si vemos:

```java
private Cliente cliente;
```

➞ Asociación.

Si vemos:

```java
private List<Encuesta> encuestas;
```

➞ Agregación.

Si vemos:

```java
class Cliente extends Usuario
```

➞ Herencia.

Si vemos:

```java
EncuestaService service = new EncuestaService();
```

➞ Dependencia.